

Assignment Report: Exploring and Testing APIs with Postman

Prepared By: Deborah Russell-Abia

1. API Choice and Base URL

For this assignment, I chose the DummyJSON API. It is a free, publicly available REST API specifically designed for testing and prototyping. I selected it because it provides a simple JWT (JSON Web Token) authentication system and supports full CRUD (Create, Read, Update, Delete) operations on a Products resource, which perfectly meets the assignment requirements.

- **Base URL:** <https://dummyjson.com>
- **Resource Used:** Products (/products)

2. Authentication Setup

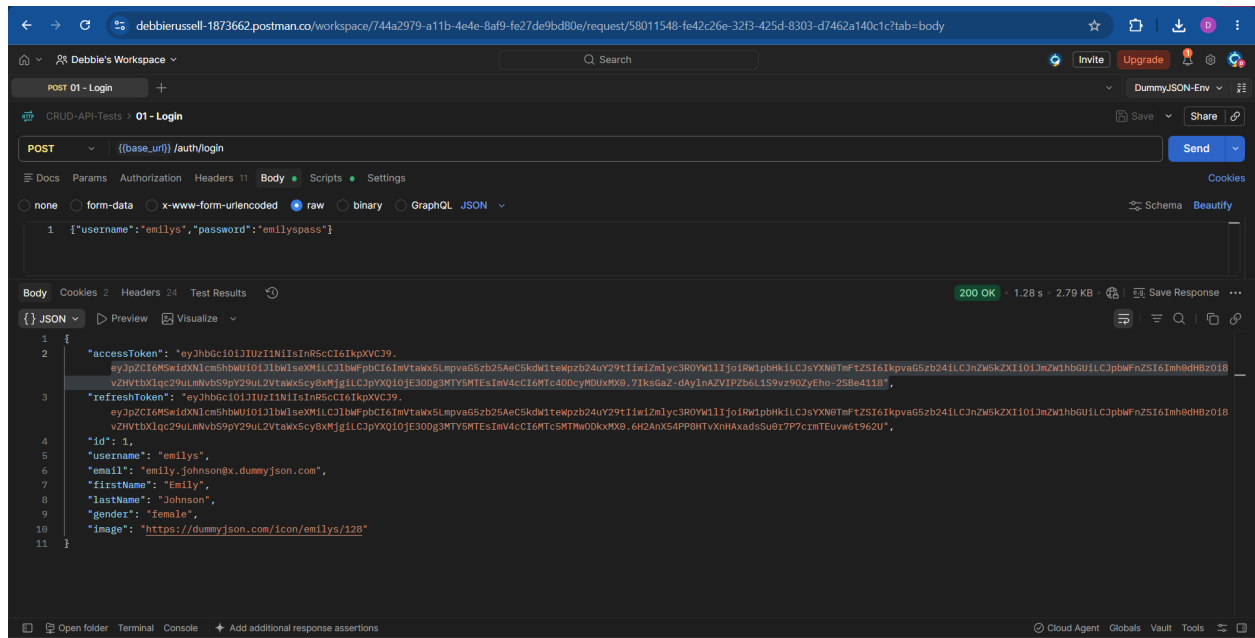
The API uses JWT Bearer Token authentication.

How it works:

I sent a POST request to the /auth/login endpoint with a valid username and password.

The server responded with a 200 OK status and a JSON body containing an accessToken.

I used a Postman script to automatically extract this token and save it into my environment variables.



Login Credentials Used:

- **Username:** emilys
- **Password:** emilyspass

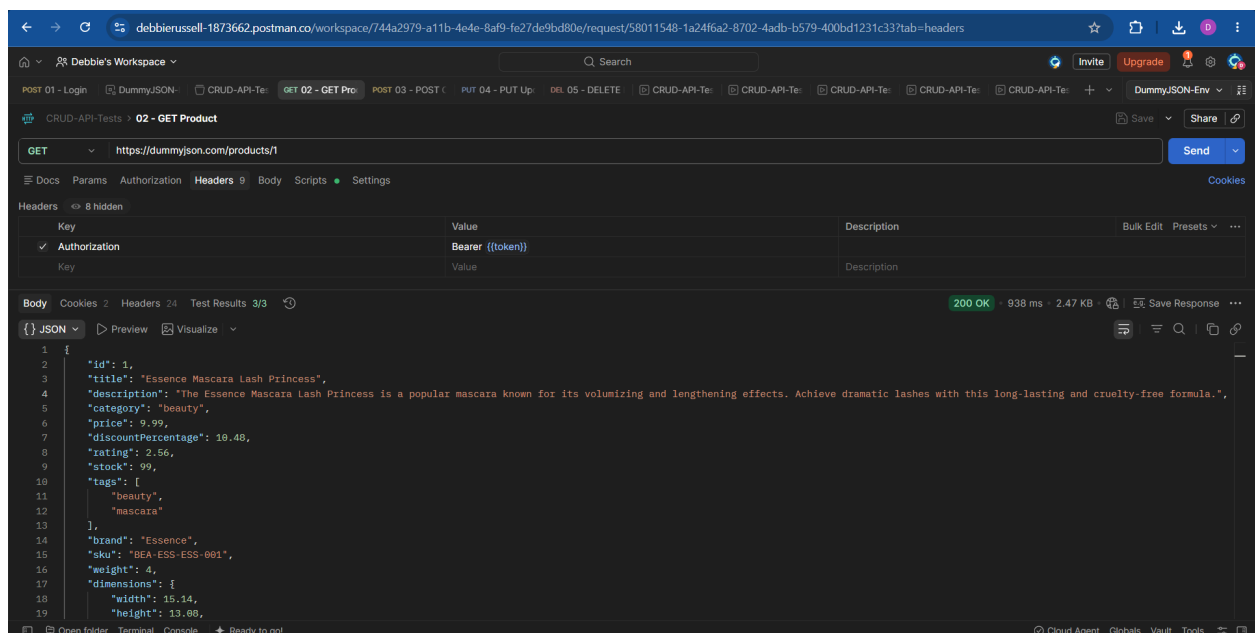
3. CRUD Operations

I successfully implemented all four CRUD operations. Below is the documentation for each endpoint, along with request/response details.

3.1 GET – Retrieve Data (Read)

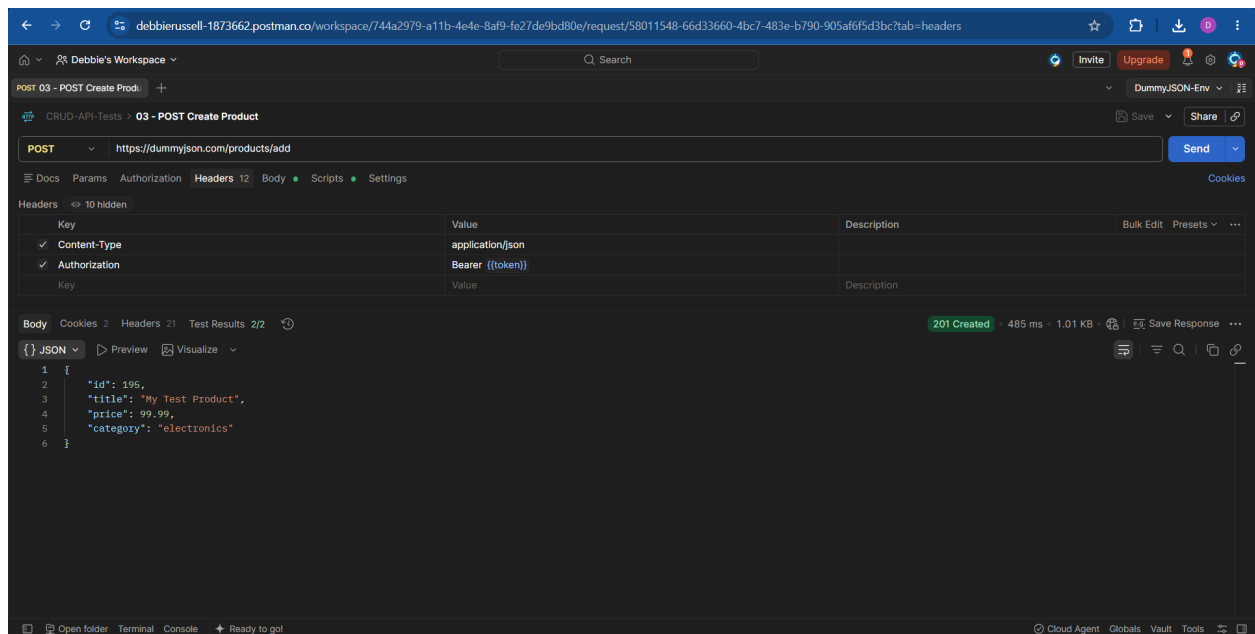
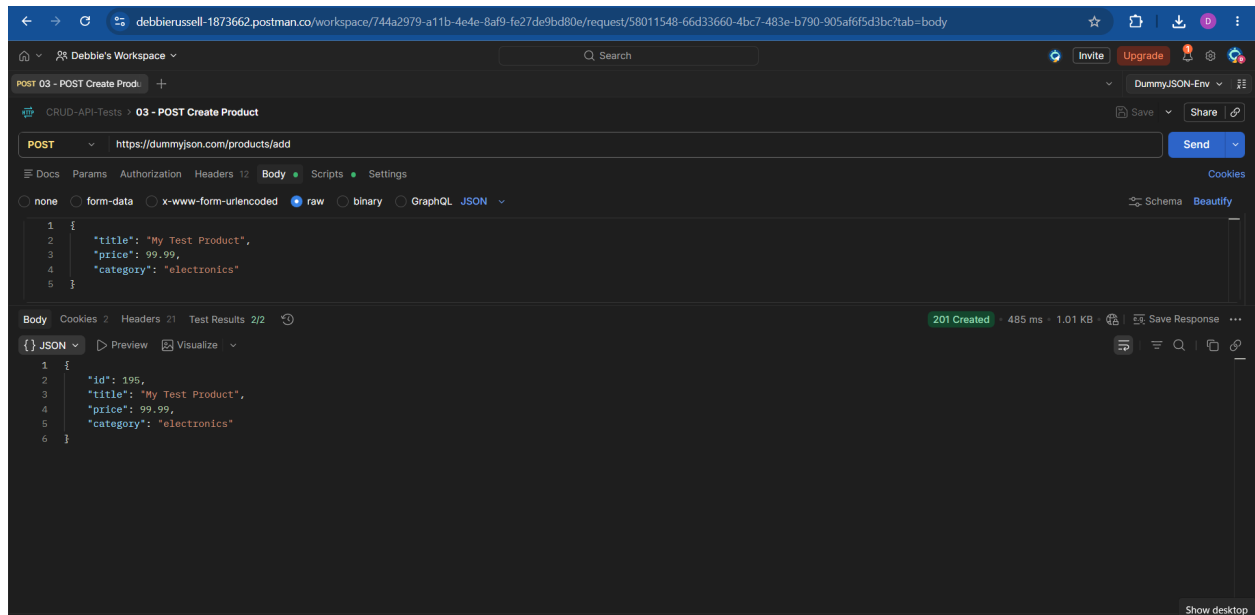
- **Endpoint:** /products/1
- **Method:** GET
- **Full URL:** https://dummyjson.com/products/1

- **Headers:** Authorization: Bearer {{token}}
- **Body:** None
- **Response:** Returns the details of the product with ID 1 (Essence Mascara).
- **Status Code:** 200 OK



3.2 POST – Create Data (Create)

- **Endpoint:** /products/add
- **Method:** POST
- **Full URL:** https://dummyjson.com/products/add
- **Headers:** Content-Type: application/json and Authorization: Bearer {{token}}



Request Body:

json

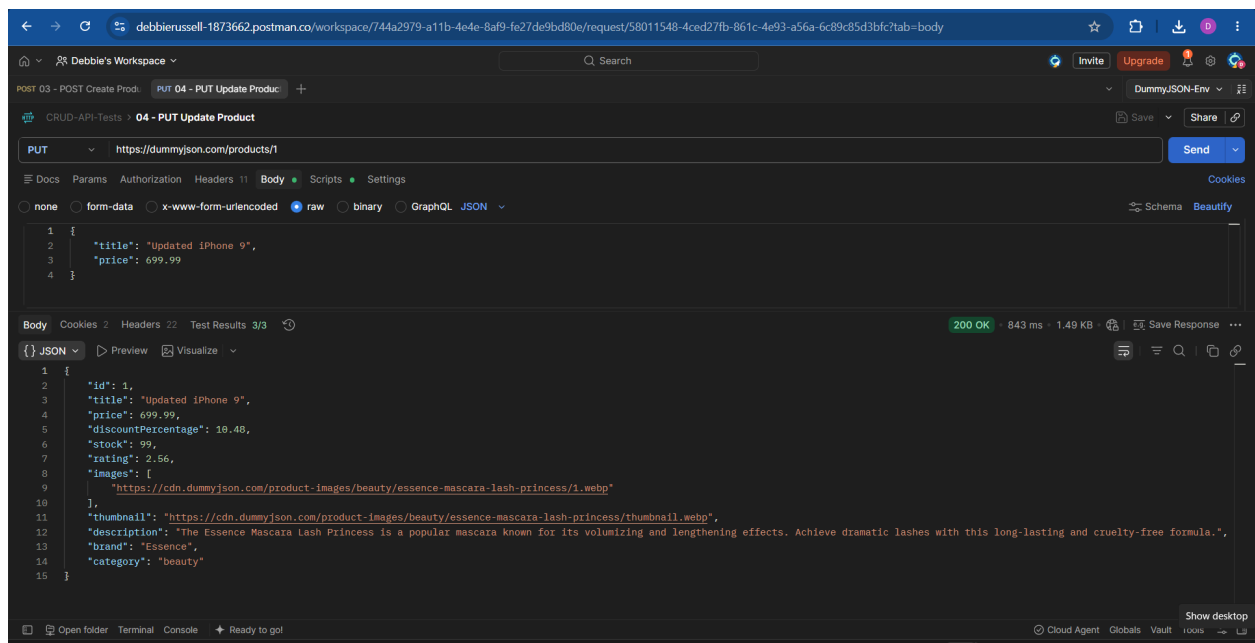
```
{  "title": "My Postman Laptop",
```

```
"description": "A powerful laptop for testing",  
"price": 999.99,  
"category": "electronics"  
}
```

- **Response:** The API returns a new product with a generated id (e.g., 195).
- **Status Code:** 201 Created

3.3 PUT – Update Data (Update)

- **Endpoint:** /products/1
- **Method:** PUT
- **Full URL:** https://dummyjson.com/products/1
- **Headers:** Content-Type: application/json and Authorization: Bearer {{token}}



Request Body:

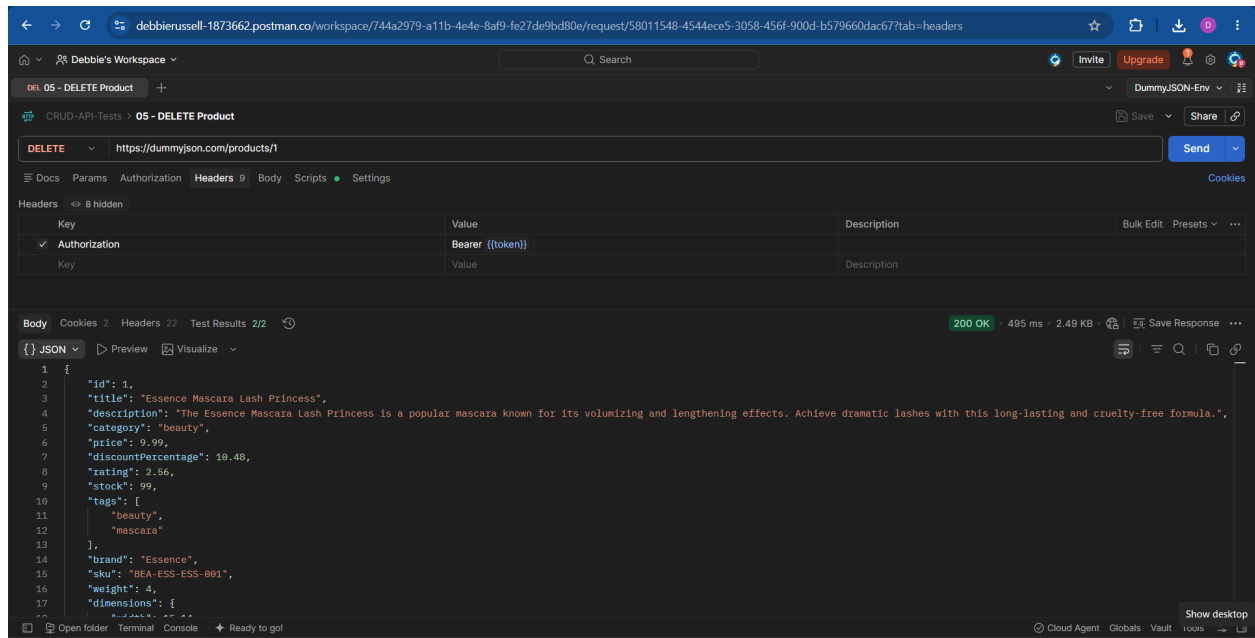
json

```
{  
  "title": "Updated iPhone 9",  
  "price": 699.99  
}
```

- **Response:** The API returns the updated product with the new title and price.
- **Status Code:** 200 OK

3.4 DELETE – Remove Data (Delete)

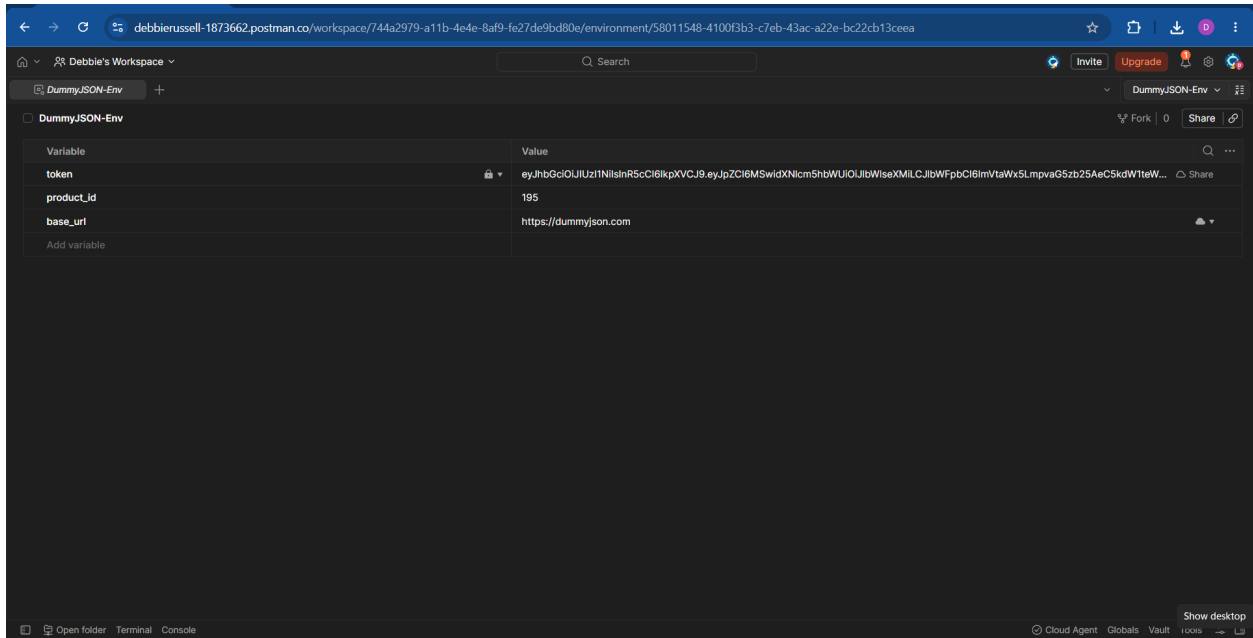
- **Endpoint:** /products/1
- **Method:** DELETE
- **Full URL:** https://dummyjson.com/products/1
- **Headers:** Authorization: Bearer {{token}}
- **Body:** None
- **Response:** The API performs a soft delete, returning "isDeleted": true and a timestamp.
- **Status Code:** 200 OK



4. Environment Variables & Automation

4.1 Environment Variables

To make the testing process dynamic and reusable, I created a Postman Environment named DummyJSON-Env with the following variables:



Variable Name	Value	Purpose
base_url	https://dummyjson.com	Stores the base URL to avoid hardcoding.
token	Auto filled	Stores the JWT access token after login.
product_id	Auto filled	Stores the ID of the newly created product.

4.2 Automation (Pre-request Script)

To automate the authentication process, I added a Pre-request Script to my CRUD-API-Tests collection. This script runs automatically before every request.

Functionality:

It checks if the token environment variable exists. If the token is missing, the script sends a login request to retrieve a new token and saves it to the environment. This ensures that the collection can run seamlessly without manual intervention.

Script Code:

javascript

```
// Auto-login if token is missing
```

```
var token = pm.environment.get("token");
```

```
if (!token || token === "") {
```

```
  pm.sendRequest({
```

```
    url: "https://dummyjson.com/auth/login",
```

```
    method: "POST",
```

```
    header: {
```

```
      "Content-Type": "application/json"
```

```
    },
```

```
    body: {
```

```
      mode: "raw",
```

```
      raw: JSON.stringify({
```

```
        username: "emilys",
```

```
        password: "emilypass"
```

```
      })
```

```
    }
```

```
  }, function (err, res) {
```

```
    if (!err) {
```

```
      var jsonData = res.json();
```

```
      if (jsonData.accessToken) {
```

```
        pm.environment.set("token", jsonData.accessToken);
```

```
        console.log("Token auto-refreshed!");
```

```
    }  
  }  
});  
}
```

5. Collection Runner Execution

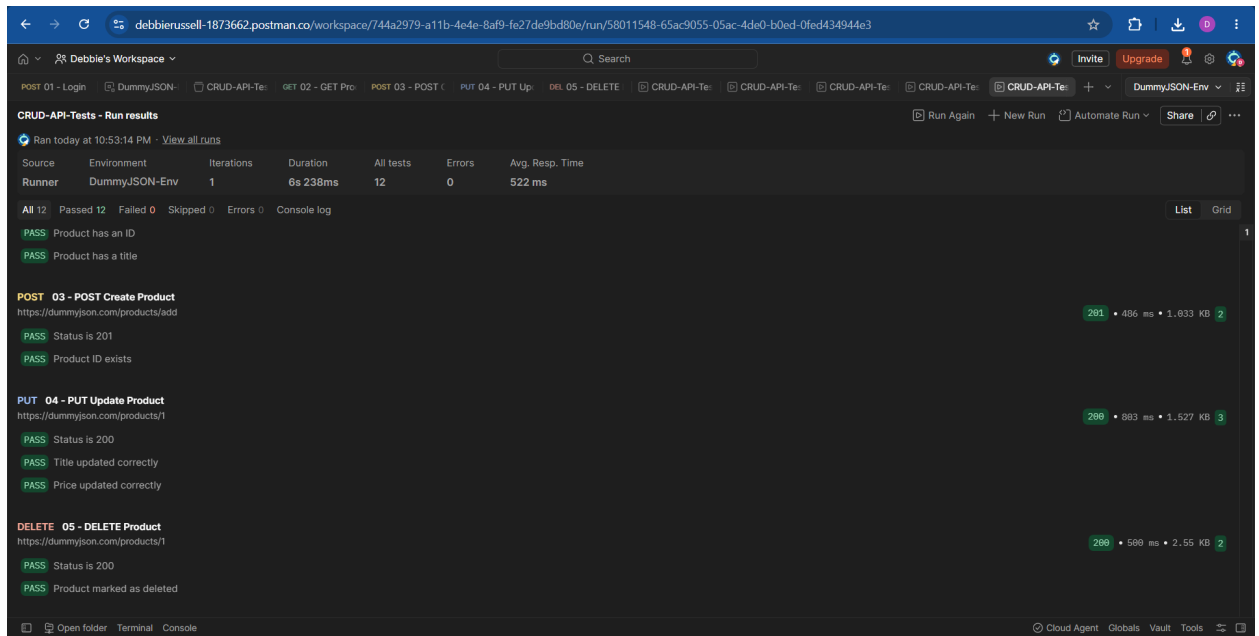
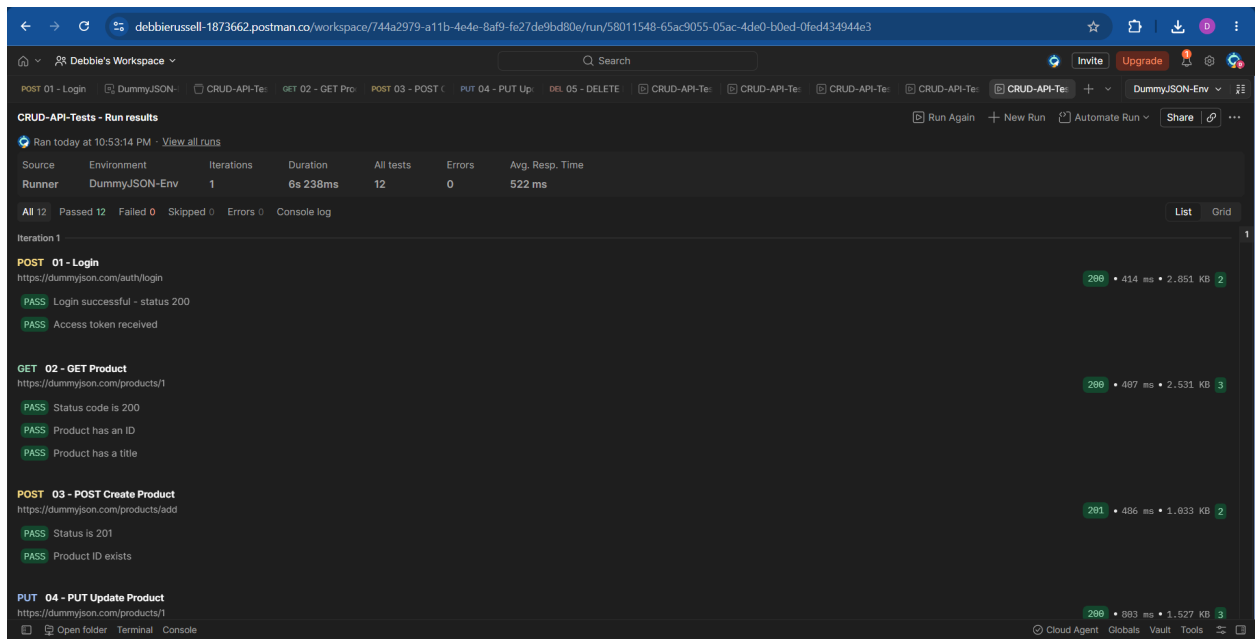
To validate the entire workflow, I ran the Collection Runner in Postman. I executed all 5 requests in sequence:

- 01 - Login
- 02 - GET Product
- 03 - POST Create Product
- 04 - PUT Update Product
- 05 - DELETE Product

I wrote test scripts for each request to verify status codes, response times, and specific data fields (e.g., token existence, product ID, updated title, isDeleted flag).

Results:

- **Total Tests:** 12
- **Passed:** 12
- **Failed:** 0



All requests passed successfully which confirmed that authentication, CRUD operations, and automation are working correctly.

6. Conclusion

In this assignment, I successfully explored and tested a public API using Postman. I implemented JWT authentication, performed full CRUD operations (Create, Read, Update, Delete), and used environment variables and pre-request scripts for automation. The Collection Runner confirmed that all tests pass which shows a complete and functional API testing workflow.

7. Submission Artifacts

The following files have been exported and uploaded to my GitHub repository:

- collection.json (Exported Postman Collection)
- environment.json (Exported Postman Environment)
- This report (PDF)

GitHub Repository Link: <https://github.com/debbierussell/postman-api-testing-assignment.git>